



LOCALIZAÇÃO

Ian Gabriel Braga Trinta

Caio Possidio Vieira



Ferramentas



Estudando os dados

```
merged_df.head(10)
```

	grupo	btsId	btsNetNome	lat_x	lon_x	cch	azimuth	ponto_id	delay_1	delay_2	delay_3	lat_y	lon_y	indoor	rssid_id	rssid_value
0	1	BTS_1_1	PEMDL01_1	-8.04342	-34.952	873	0	6604	5.0	3.0	2.0	-8.055786	-34.951505	True	rssid_1_1	-103.705000
1	1	BTS_1_1	PEMDL01_1	-8.04342	-34.952	873	0	5515	5.0	3.0	2.0	-8.055784	-34.951608	True	rssid_1_1	-105.225000
2	1	BTS_1_1	PEMDL01_1	-8.04342	-34.952	873	0	5049	5.0	3.0	2.0	-8.055506	-34.951693	True	rssid_1_1	-102.610000
3	1	BTS_1_1	PEMDL01_1	-8.04342	-34.952	873	0	8737	4.0	7.0	2.0	-8.050390	-34.959630	False	rssid_1_1	-89.140000
4	1	BTS_1_1	PEMDL01_1	-8.04342	-34.952	873	0	1203	5.0	3.0	2.0	-8.055380	-34.951736	True	rssid_1_1	-109.088333
5	1	BTS_1_1	PEMDL01_1	-8.04342	-34.952	873	0	6952	6.0	5.0	0.0	-8.055780	-34.956390	False	rssid_1_1	-91.530000
6	1	BTS_1_1	PEMDL01_1	-8.04342	-34.952	873	0	8875	3.0	4.0	3.0	-8.049710	-34.954640	False	rssid_1_1	-80.246429
7	1	BTS_1_1	PEMDL01_1	-8.04342	-34.952	873	0	1926	5.0	3.0	3.0	-8.055494	-34.951316	True	rssid_1_1	-98.748000
8	1	BTS_1_1	PEMDL01_1	-8.04342	-34.952	873	0	2641	5.0	3.0	3.0	-8.055684	-34.951331	True	rssid_1_1	-115.000000
9	1	BTS_1_1	PEMDL01_1	-8.04342	-34.952	873	0	3399	5.0	3.0	3.0	-8.055834	-34.951362	True	rssid_1_1	-100.830000

```
merged_df_test.head(10)
```

	grupo	btsId	btsNetNome	lat	lon	cch	azimuth	ponto_id	delay_1	delay_2	delay_3	rssid_id	rssid_value
0	1	BTS_1_1	PEMDL01_1	-8.04342	-34.952	873	0	0	5.0	3.0	2.0	rssid_1_1	-115.0
1	1	BTS_1_1	PEMDL01_1	-8.04342	-34.952	873	0	3	5.0	3.0	2.0	rssid_1_1	-115.0
2	1	BTS_1_1	PEMDL01_1	-8.04342	-34.952	873	0	8	5.0	3.0	2.0	rssid_1_1	-115.0
3	1	BTS_1_1	PEMDL01_1	-8.04342	-34.952	873	0	10	5.0	3.0	2.0	rssid_1_1	-115.0
4	1	BTS_1_1	PEMDL01_1	-8.04342	-34.952	873	0	12	5.0	3.0	2.0	rssid_1_1	-115.0
5	1	BTS_1_1	PEMDL01_1	-8.04342	-34.952	873	0	14	5.0	3.0	2.0	rssid_1_1	-115.0
6	1	BTS_1_1	PEMDL01_1	-8.04342	-34.952	873	0	17	5.0	3.0	2.0	rssid_1_1	-115.0
7	1	BTS_1_1	PEMDL01_1	-8.04342	-34.952	873	0	19	5.0	3.0	2.0	rssid_1_1	-115.0
8	1	BTS_1_1	PEMDL01_1	-8.04342	-34.952	873	0	23	5.0	3.0	2.0	rssid_1_1	-115.0
9	1	BTS_1_1	PEMDL01_1	-8.04342	-34.952	873	0	29	5.0	3.0	2.0	rssid_1_1	-115.0

```
rssid_id_counts = merged_df['rssid_id'].value_counts()
```

```
# Display the counts
print(rssid_id_counts)
```

```
rssid_id
rssid_1_1    6775
rssid_1_2    6775
rssid_1_3    6775
rssid_2_1    6775
rssid_2_2    6775
rssid_2_3    6775
rssid_3_1    6775
rssid_3_2    6775
rssid_3_3    6775
Name: count, dtype: int64
```

```
rssid_id_counts_test = merged_df_test['rssid_id'].value_counts()
```

```
# Display the counts
print(rssid_id_counts_test)
```

```
rssid_id
rssid_1_1    2904
rssid_1_2    2904
rssid_1_3    2904
rssid_2_1    2904
rssid_2_2    2904
rssid_2_3    2904
rssid_3_1    2904
rssid_3_2    2904
rssid_3_3    2904
Name: count, dtype: int64
```



Tratamento de dados

```
merged_df_test.isnull().sum()
```

```
grupo      0  
btsId      0  
btsNetNome 0  
lat        0  
lon        0  
cch        0  
azimuth    0  
ponto_id   0  
delay_1    0  
delay_2    0  
delay_3    0  
rssi_id    0  
rssi_value 0  
dtype: int64
```

```
merged_df.isnull().sum()
```

```
[87]  
... grupo      0  
    btsId      0  
    btsNetNome 0  
    lat_x      0  
    lon_x      0  
    cch        0  
    azimuth    0  
    ponto_id   0  
    delay_1    0  
    delay_2    0  
    delay_3    0  
    lat_y      0  
    lon_y      0  
    indoor     0  
    rssi_id    0  
    rssi_value 0  
    dtype: int64
```

Number of outliers detected: 590

```
merged_df.duplicated().sum()
```

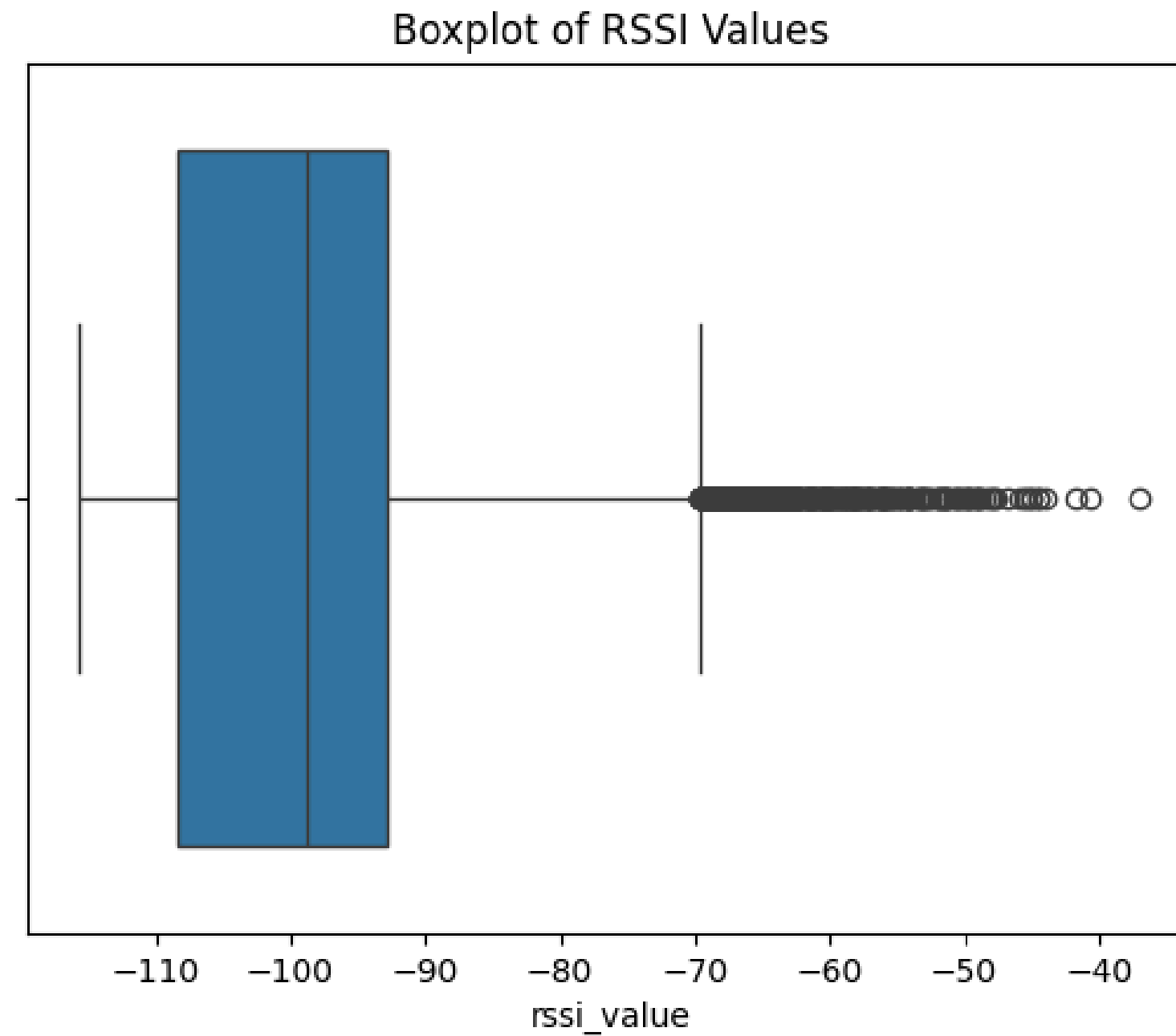
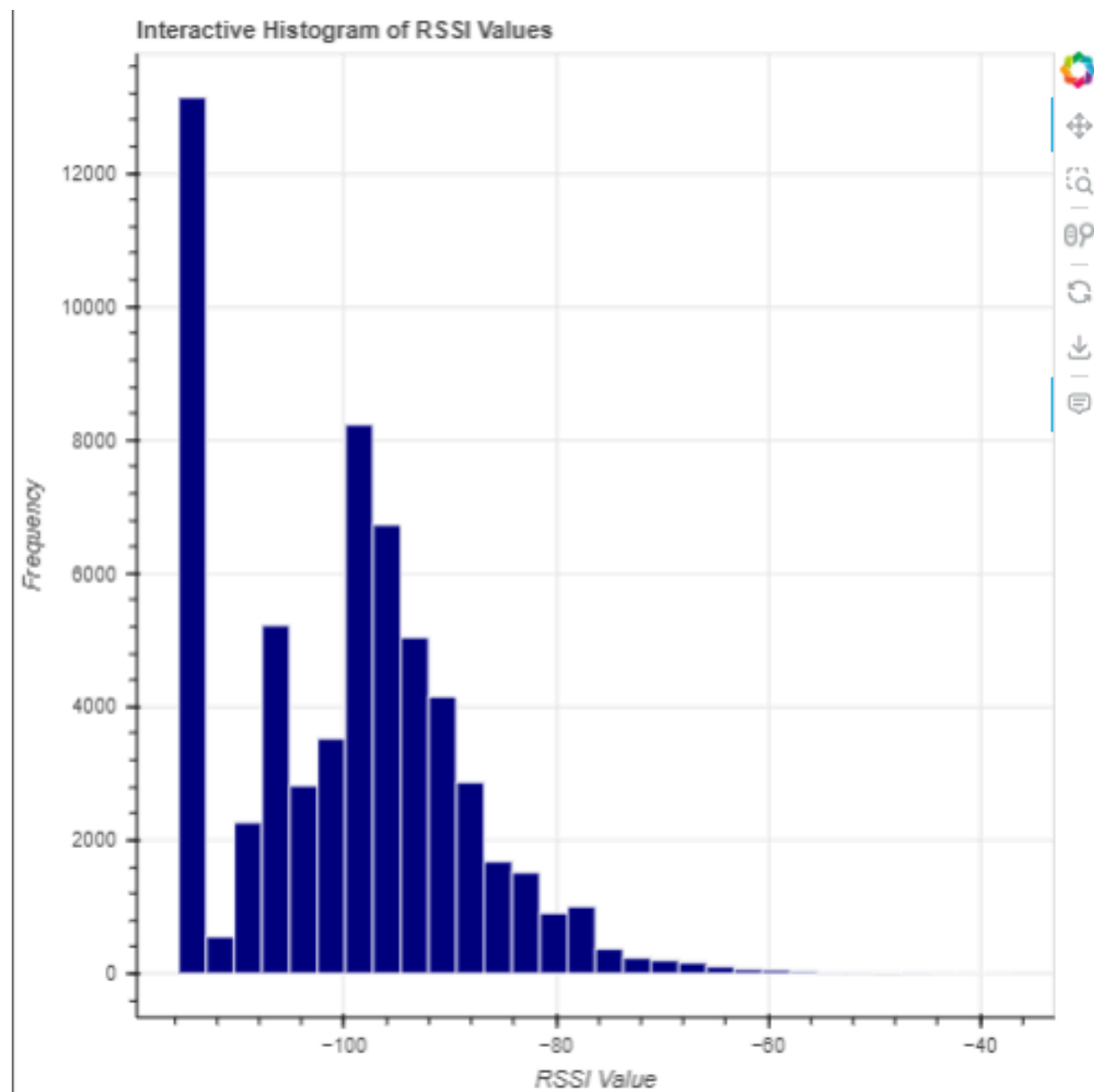
```
[89]  
... 0
```

```
merged_df_test.duplicated().sum()
```

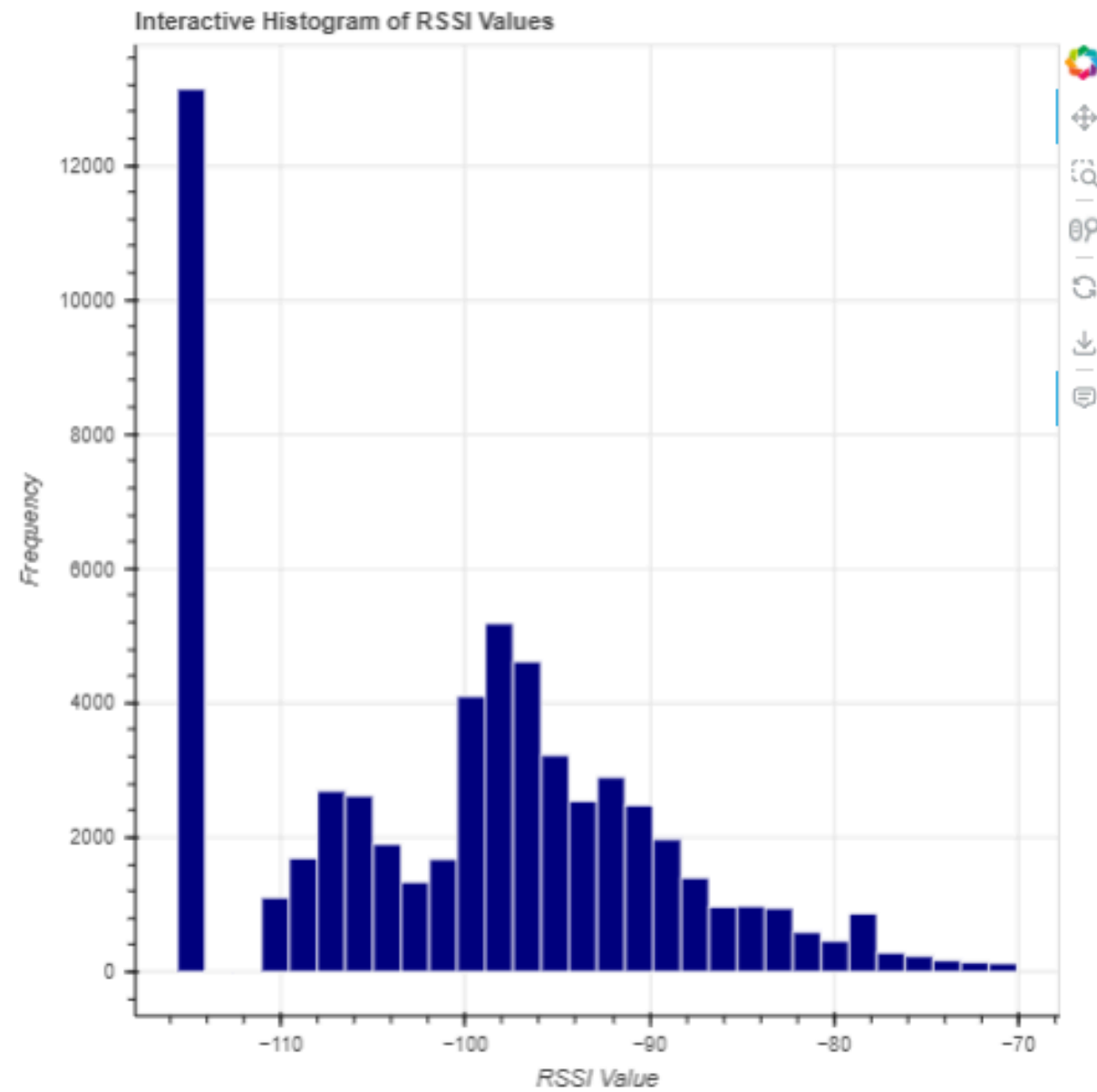
```
[90]  
... 0
```



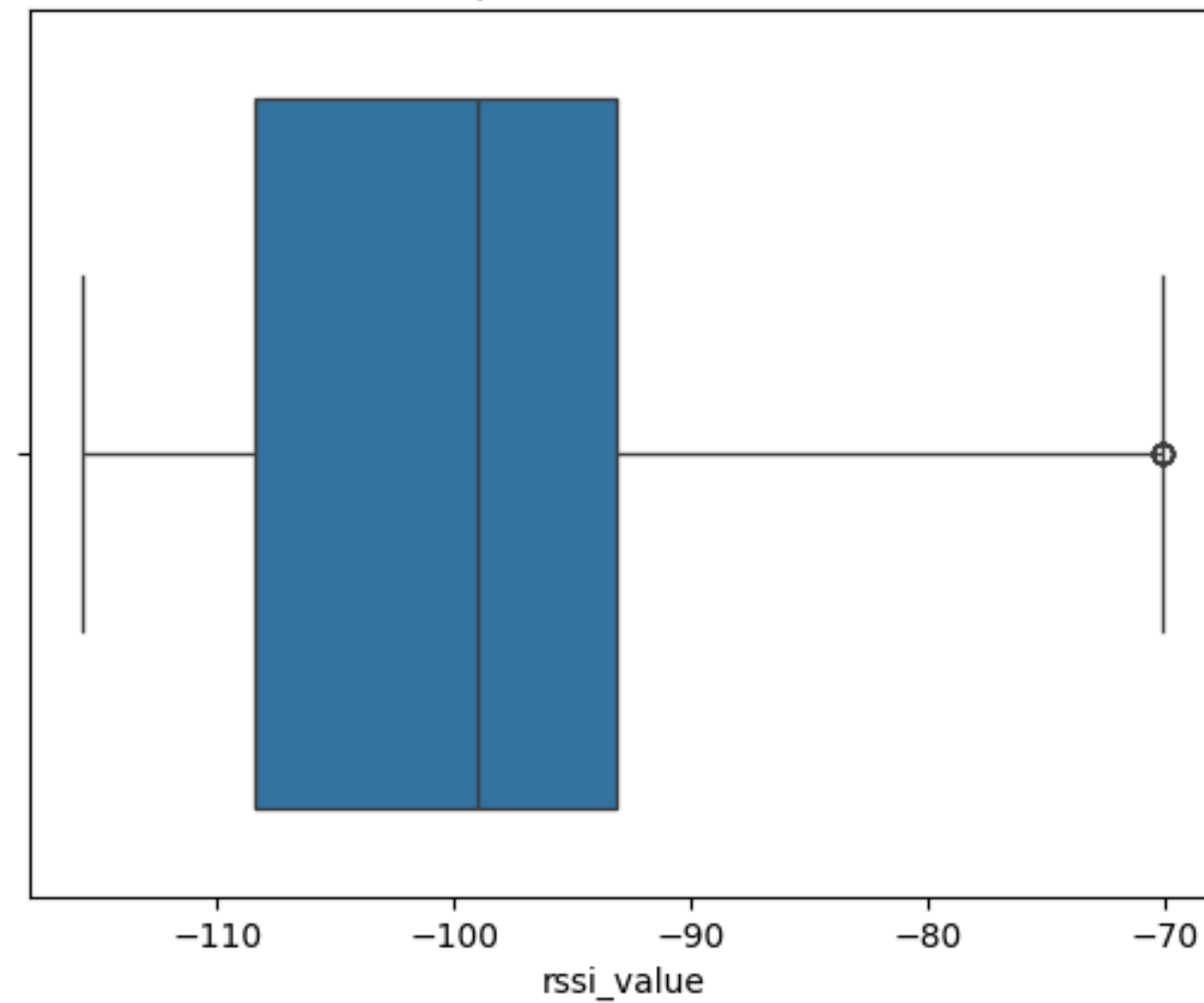
Remoção de outliers



Remoção de outliers



Boxplot of RSSI Values



Normalização

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler

scaler = MinMaxScaler()
#scaler = StandardScaler()
columns_to_normalize = ['rssi_value']
merged_df[columns_to_normalize] = scaler.fit_transform(merged_df[columns_to_normalize])
merged_df.to_csv('./data/train_normalized_dataset.csv', index=False)

merged_df.head()
```

	grupo	btsId	btsNetNome	lat_x	lon_x	cch	azimuth	ponto_id	delay_1	delay_2	delay_3	lat_y	lon_y	indoor	rssi_id	rssi_value
0	1	BTS_1_1	PEMDL01_1	-8.04342	-34.952	873	0	6604	5.0	3.0	2.0	-8.055786	-34.951505	True	rssi_1_1	0.261743
1	1	BTS_1_1	PEMDL01_1	-8.04342	-34.952	873	0	5515	5.0	3.0	2.0	-8.055784	-34.951608	True	rssi_1_1	0.228380
2	1	BTS_1_1	PEMDL01_1	-8.04342	-34.952	873	0	5049	5.0	3.0	2.0	-8.055506	-34.951693	True	rssi_1_1	0.285777
3	1	BTS_1_1	PEMDL01_1	-8.04342	-34.952	873	0	8737	4.0	7.0	2.0	-8.050390	-34.959630	False	rssi_1_1	0.581431
4	1	BTS_1_1	PEMDL01_1	-8.04342	-34.952	873	0	1203	5.0	3.0	2.0	-8.055380	-34.951736	True	rssi_1_1	0.143584

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler

scaler = MinMaxScaler()

columns_to_normalize = ['rssi_value']
merged_df_test[columns_to_normalize] = scaler.fit_transform(merged_df_test[columns_to_normalize])
merged_df_test.to_csv('./data/test_normalized_dataset.csv', index=False)

merged_df_test.head()
```



Geração de Dados Sintéticos

THE SYNTHETIC DATA VAULT



```
# Definir o modelo CTGAN
model = CTGAN()

# Treinar o modelo
model.fit(data, epochs=500)

# Gerar novos dados sintéticos
new_data = model.sample(30000)

# Mostrar os primeiros registros dos dados gerados
print(new_data.head())

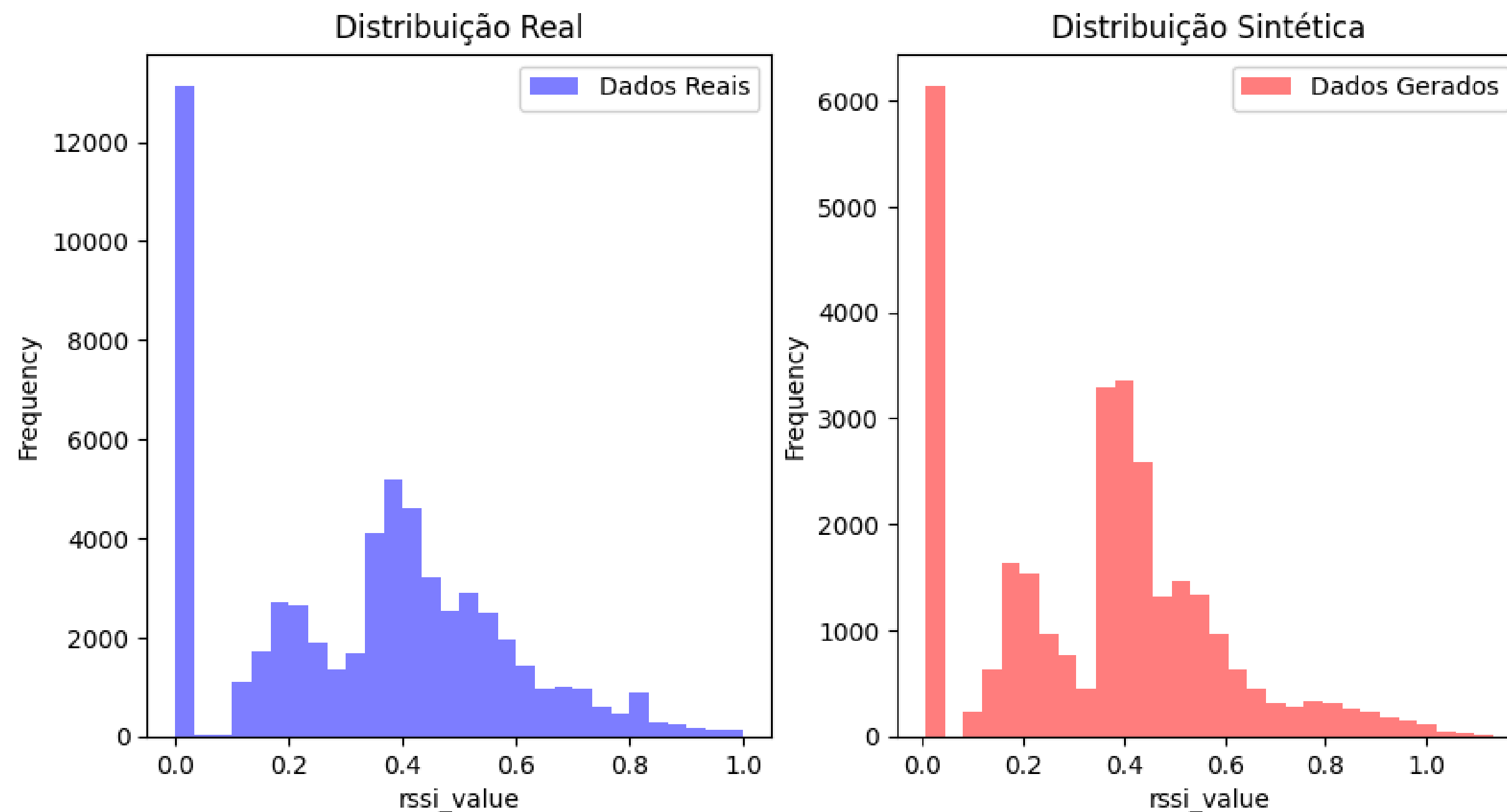
# salvando os dados gerados
new_data.to_csv('data/synthetic_data.csv', index=False)
```

[27] ✓ 41m 15.9s

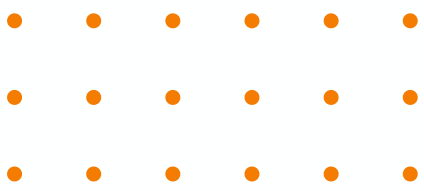
...	grupo	cch	azimuth	ponto_id	delay_1	delay_2	delay_3	lat	\
0	1	872	120	4910	4.999612	2.999441	1.999683	-8.055729	
1	1	875	240	3396	4.999964	3.000168	2.998894	-8.055754	
2	2	874	120	4385	5.000587	3.000172	1.999799	-8.054985	
3	1	872	120	7906	3.004024	2.998630	4.994818	-8.050165	
4	3	869	240	2845	4.999331	2.999481	2.999859	-8.055938	

	lon	rss_value
0	-34.951811	0.014321
1	-34.951985	0.008045
2	-34.951910	0.241099
3	-34.947382	0.530870
4	-34.951283	0.014218

Geração de Dados Sintéticos



KNN



```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_neighbors': range(1, 21),
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan', 'minkowski']
}

# Initialize the KNN model
knn = KNeighborsRegressor()

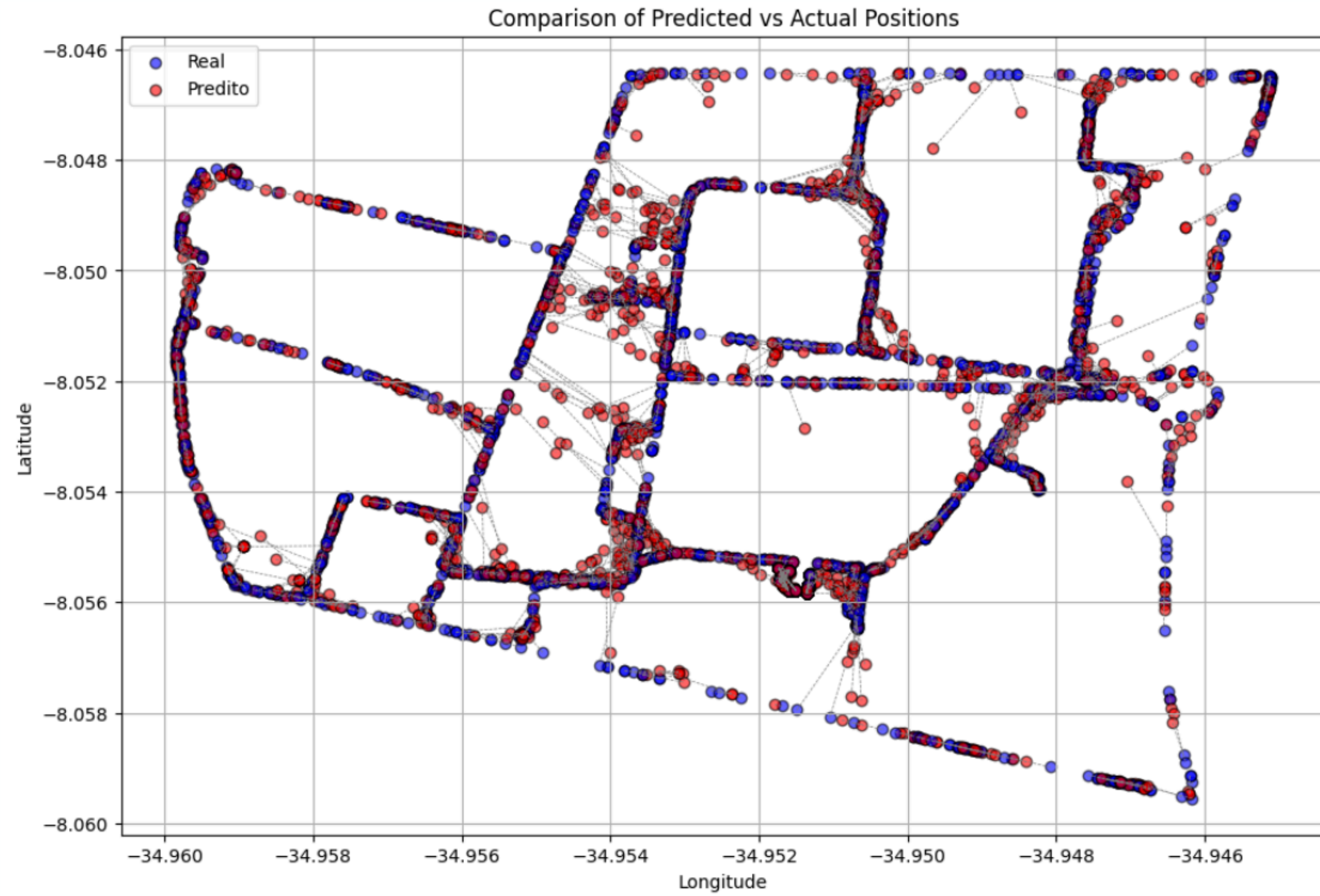
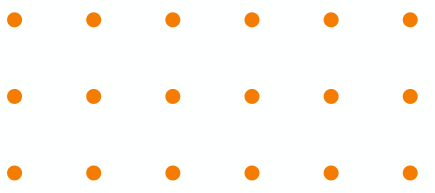
# Set up GridSearchCV
grid_search = GridSearchCV(estimator=knn, param_grid=param_grid, cv=5, scoring='neg_mean_squared_error', verbose=2)

# Fit the grid search to the data
grid_search.fit(X_train, y_train)
```

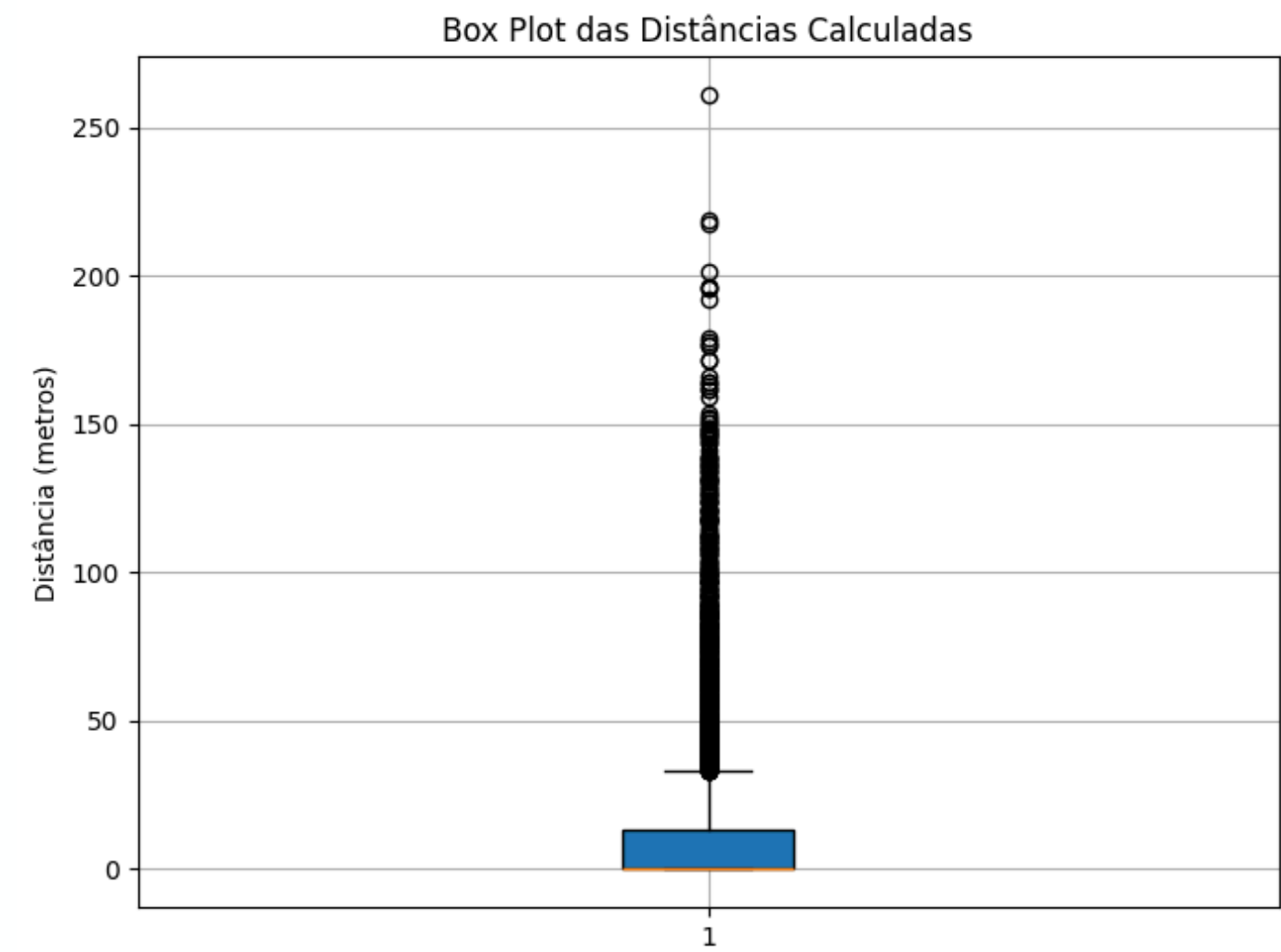
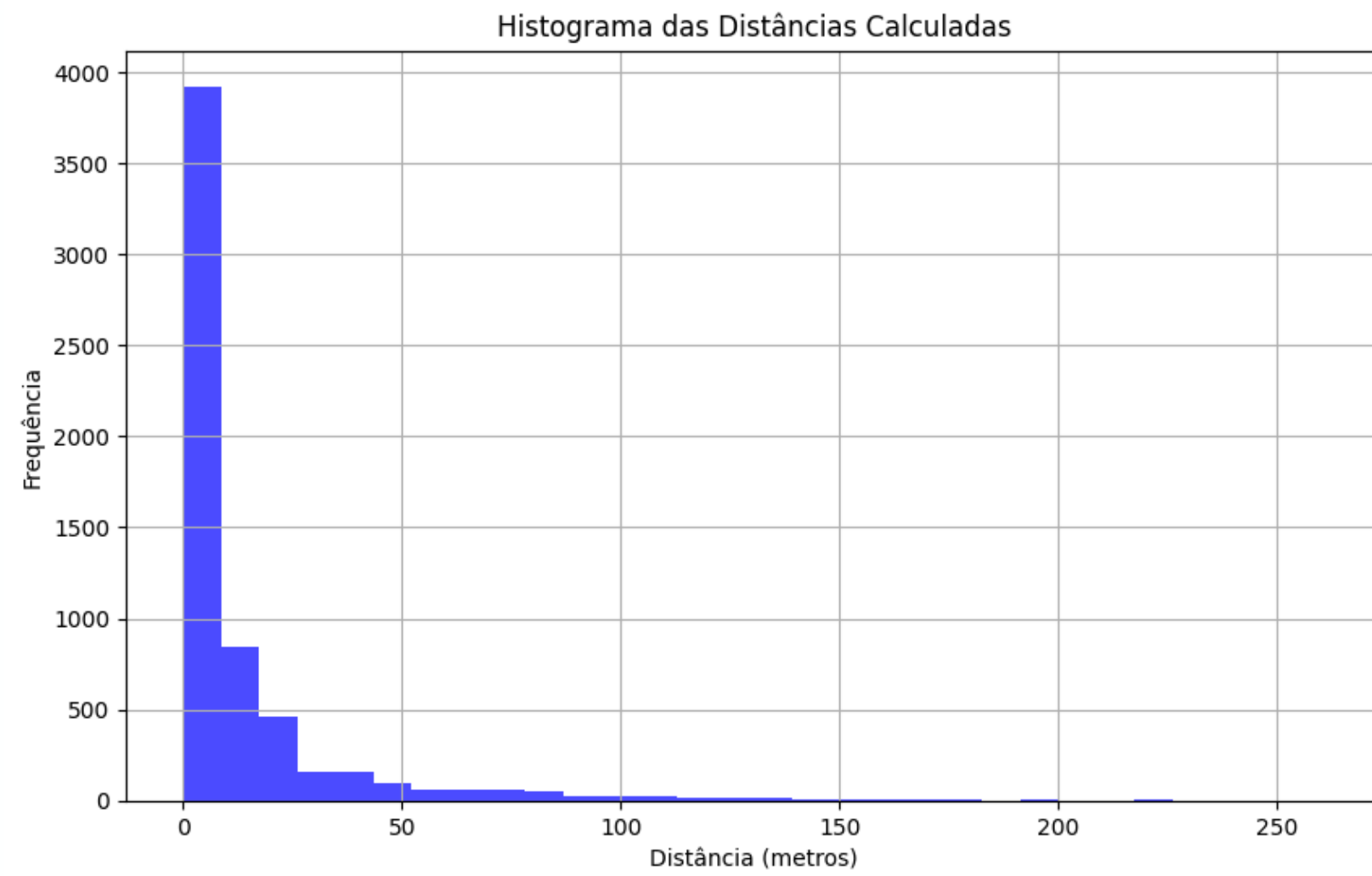
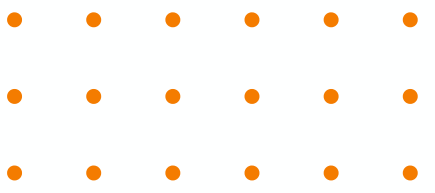
Erro mínimo na distância(em metros): 0.0
Erro médio na distância(em metros): 12.534230722957988
Erro máximo na distância(em metros): 260.82245052819627
Desvio padrão do erro na distância(em metros): 25.07931044824998

MAE: 7.092146485882531e-05
MSE: 3.206010082924566e-08
Accuracy: 0.9939824065725464

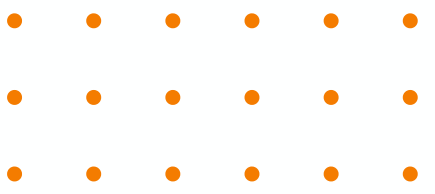
KNN



KNN



Random Forest



```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators': [200, 300, 400, 500],
    'max_depth': [10, 20, 30, 40, 50],
    'min_samples_split': [2, 5, 10],
}

# Chamando a função do Random Forest
random_forest = RandomForestRegressor()

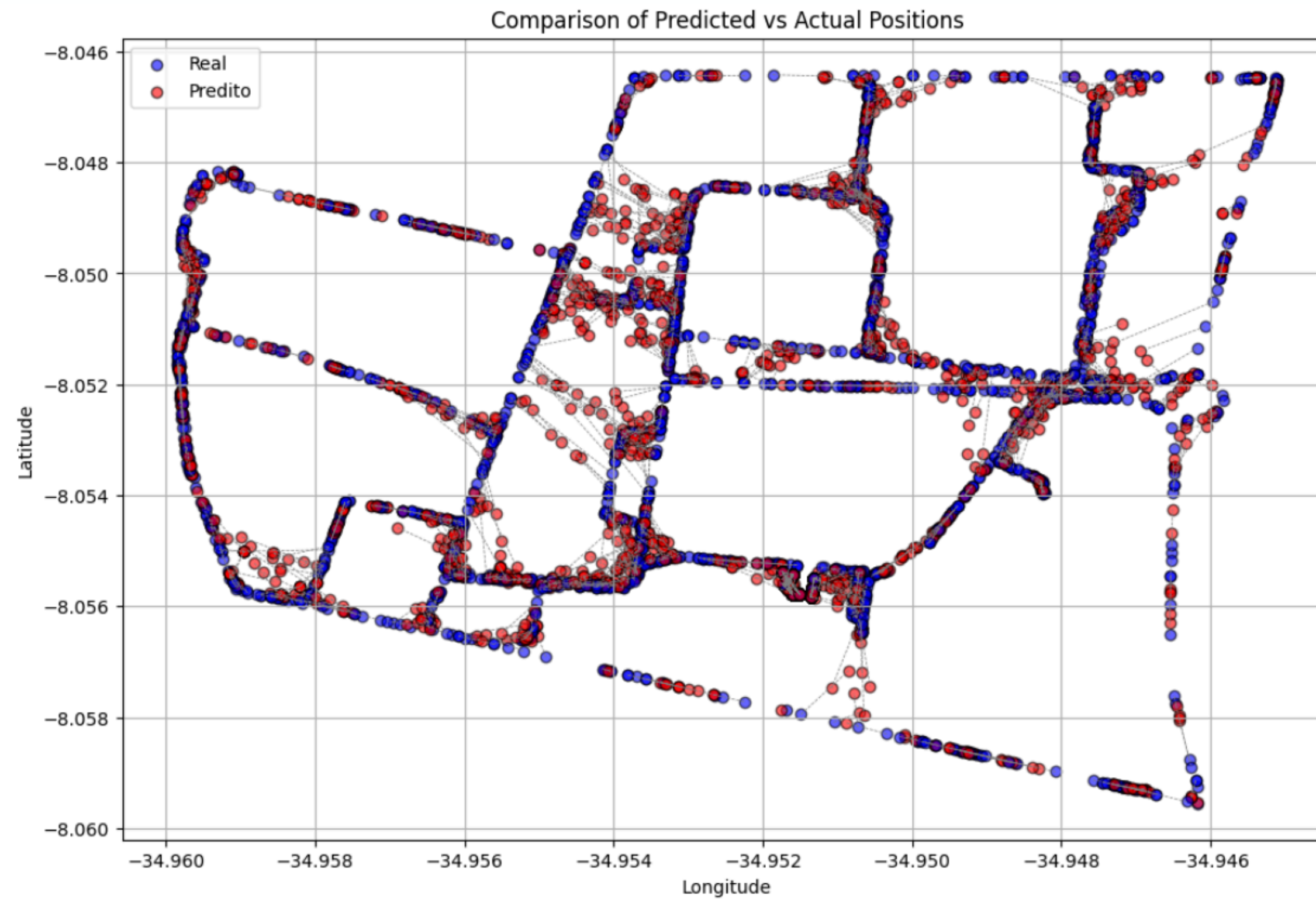
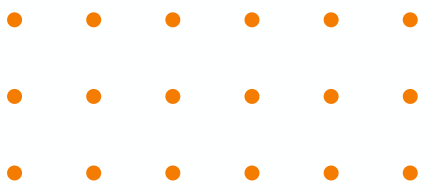
# Grid search
grid_search = GridSearchCV(estimator=random_forest, param_grid=param_grid, cv=5, scoring='neg_mean_squared_error', verbose=2)

# Fit do modelo
grid_search.fit(X_train, y_train)
```

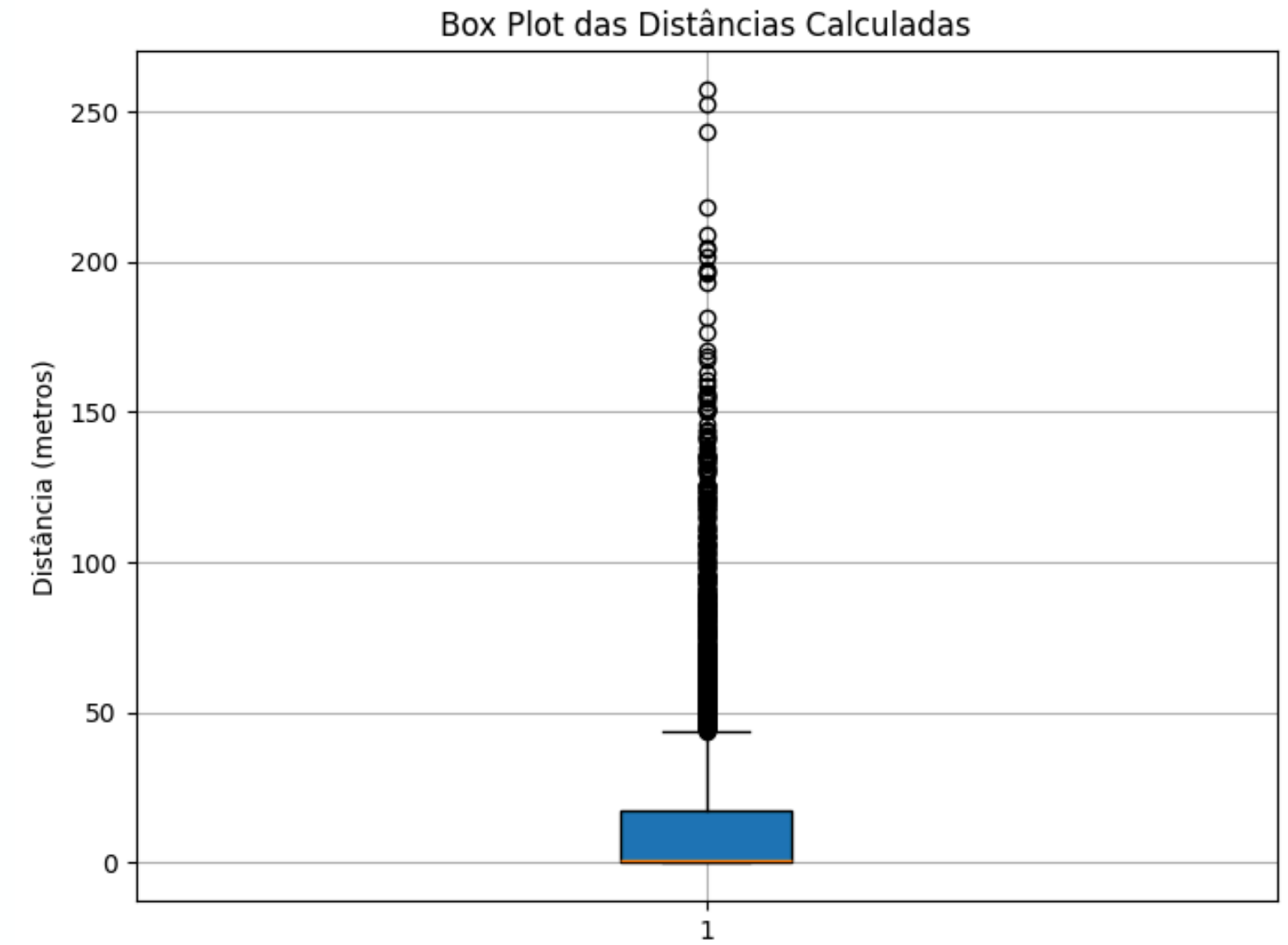
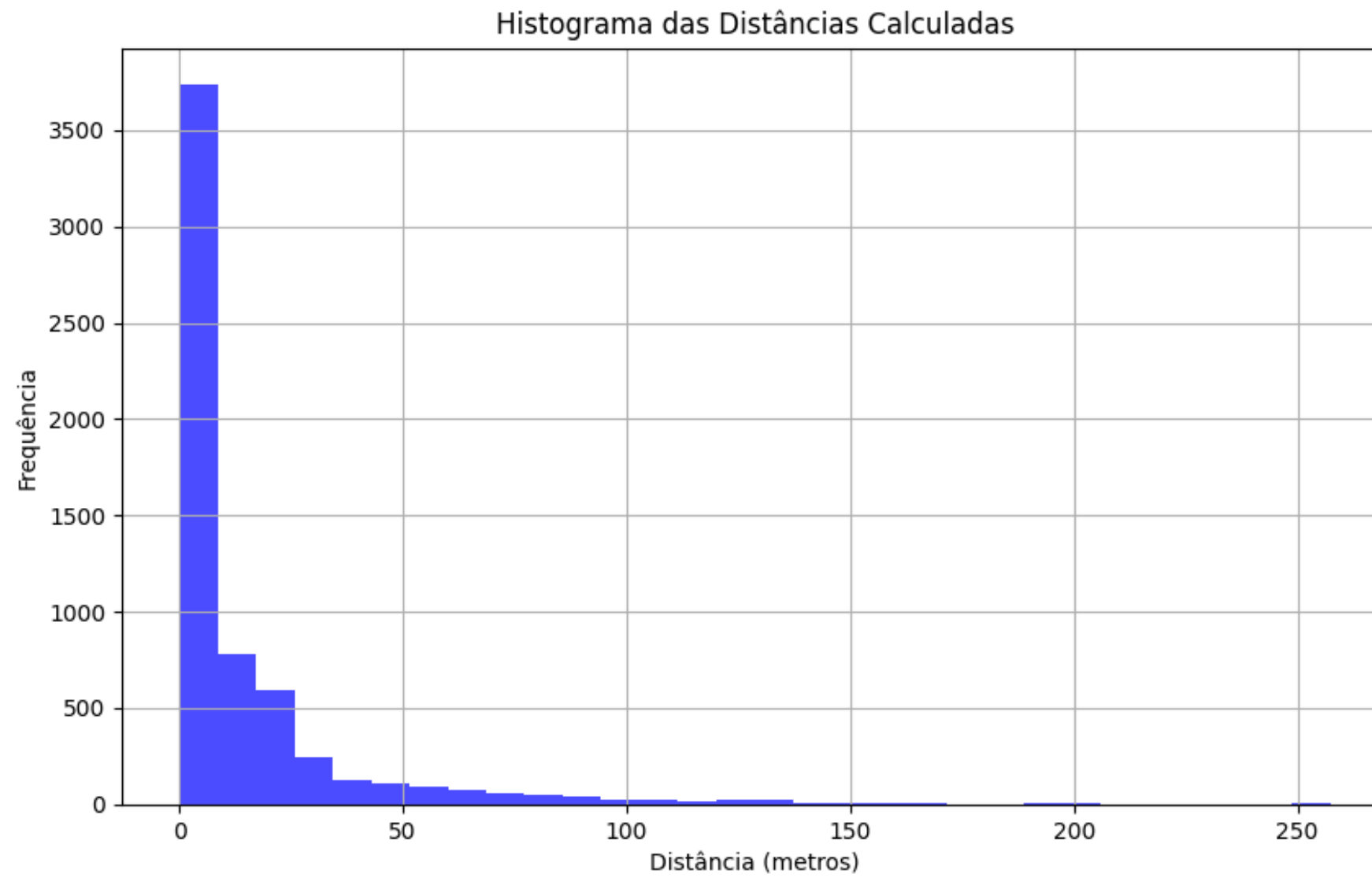
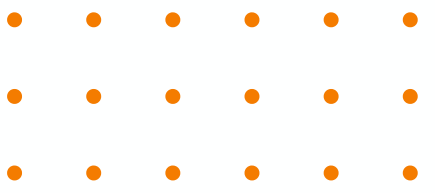
Erro mínimo na distância(em metros): 1.4146461779773745e-09
Erro médio na distância(em metros): 13.730352683107908
Erro máximo na distância(em metros): 257.360843072137
Desvio padrão do erro na distância(em metros): 25.858387868888535

MAE: 7.822344436864134e-05
MSE: 3.4972078869761934e-08
Accuracy: 0.9934179270324579

Random Forest



Random Forest



DIFICULDADES



Escalabilidade da
geração de
imagens sintéticas



Tempo de
treinamento muito
alto no GridSearch da
Random Forest



O resultado do treinamento
do modelo com os dados
sintéticos apresentou
métricas muito piores